

SOFTWARE DESIGN DOCUMENT

Student Attendance & Performance Tracker

Android application for administrator, faculty and student academic workflows

Team member	Roll number
R. Nirmal	231801370002
L. Nitish Kumar	231801370009
T. Veera Siva Krishna	231801370025
N. Laasya	231801370046
S. Jayanth Kumar	231801370051
S. Sai Vignesh	231801370052
T. Sai Devanand	231801370070

Document control	Value
Document ID	CUTM-SDD-SAPT-VER-0.1
Version / date	0.1 - review draft / 1 October 2026
Requirements baseline	CUTM-SRS-SAPT-VER-0.1
Backend	Google Sheets + Google Apps Script

The supplied TeamPulse SDD is a structural reference. Project design and implementation status are based on Student Tracker source and documentation.

Table of contents

Section	Page
1. Introduction	3
2. System overview	4
3. Architecture design	5-7
4. Component design	8-9
5. Data design	10-12
6. Interface design	13-16
7.1 Sign-in and role-resolution sequence	17-20
8. Security design	21
9. Error handling and reliability	22
10. Deployment and configuration	23-24
11. Testing and verification	25
12. Requirements traceability	26-27
13. Limitations and future enhancements	28
14. Conclusion and references	29
Appendix A. Source component index	30

List of figures

Figure	Page
3.5 System architecture	6
3.6 Component dependency view	7
5.8 Logical entity relationship diagram	12
6.11 Role-based navigation diagram	16
7.1 Sign-in and role-resolution sequence	17
7.2 Attendance runtime flowchart	18
7.3 Assessment runtime flowchart	19
7.4 Attendance and assessment state transitions	20
10.5 Deployment topology	24

1. Introduction

1.1 Purpose

This Software Design Document explains how Student Tracker implements the behavior described in its SRS. It records the application architecture, component responsibilities, data model, request contracts, runtime flows, security boundaries, deployment and verification strategy.

1.2 Scope

The baseline covers Google-authenticated administrator management of students, faculty, subjects and enrollment; assigned staff attendance and assessment workflows; personal student progress; scoped attendance reports; and audit-backed storage.

The Android client uses Kotlin and Jetpack Compose. The backend is JavaScript deployed as Google Apps Script with Google Sheets storage. Room, SQLCipher, Hilt, WorkManager, Firebase and a full MVVM/repository layer are not part of the current baseline.

1.3 Intended audience

Project evaluators, faculty reviewers, the seven-member team, maintainers, testers and institution administrators. This document supports design review and presentation; it does not certify a production release.

1.4 Definitions and abbreviations

Term	Meaning
SDD / SRS	Software Design Document / Software Requirements Specification.
API / UI	Application Programming Interface / User Interface.
Principal	Server-approved account identity, role, profile and expiry.
Revision	Record version used to detect stale edits.
Idempotent retry	Repeating an identical supported request yields one logical result.
Journal / projection	Append-only version history / reconstructed latest record state.
CSCI	Computer Software Configuration Item.

2. System overview

2.1 Product description

Student Tracker maintains a shared academic record through three user experiences. Staff submit attendance for enrolled students and publish complete assessment marks. Students view their own attendance totals and published results. Administrators prepare profiles, subjects and rosters.

2.2 User groups

User	Design responsibility
Administrator	Dashboard counts; profile/subject CRUD; enrollment; academic workflows and reports.
Faculty	Assigned-subject hub; current roster; attendance, assessment drafts/publication and scoped reports.
Student	Own attendance summaries and published marks; refresh and logout.
Operator / maintainer	Cloud configuration, deployment, allowlist provisioning, direct storage controls and eventual backups/monitoring.

2.3 Operating environment

Item	Observed configuration
Application ID / namespace	com.example.studenttracker
Language / UI	Kotlin / Jetpack Compose + Material 3
Android SDK	Minimum 24; target 37; compile 37
Build JVM / compatibility	JDK 21 for build; Java source/target compatibility 11
Networking	Retrofit 2.11.0; OkHttp 4.12.0; Gson
Local persistence	Preferences DataStore metadata; temporary report files
Backend / storage	Apps Script V8 / two private Google Sheets tabs

Current academic operations require connectivity. A fresh app process requires sign-in because tokens are not persisted. Durable offline drafts and automatic synchronization remain planned.

3. Architecture design

3.1 Architectural overview

The design separates presentation packages, network contracts/session state, server domain logic and the spreadsheet adapter. The current client is screen-oriented: Compose screens invoke suspend API methods directly, usually through RemoteContent or a screen coroutine. Server domain rules are isolated in createService and tested with a store abstraction.

3.2 Architectural style

Layer / boundary	Responsibility / source
Android entry and navigation	MainActivity, StudentTrackerApp and AppNavigation choose the verified role graph.
Presentation	Feature screens and shared WorkspacePage, RemoteContent, InfoCard and guardedBack.
Network / identity	GoogleSheetsApi models, NetworkModule, TrackerInterceptor and AuthSession.
Local metadata / export	UserPreferences and CsvExporter; FileProvider cache export sharing.
Backend authentication	auth.js verifies RSA signatures; core.js validates claims and allowlist identity.
Backend domain	createService applies CRUD, enrollment, attendance, assessments and report rules.
Cloud adapter	apps-script.js dispatches HTTP, obtains certificates, locks and reads/appends Sheets.

3.3 Asynchronous work and dependencies

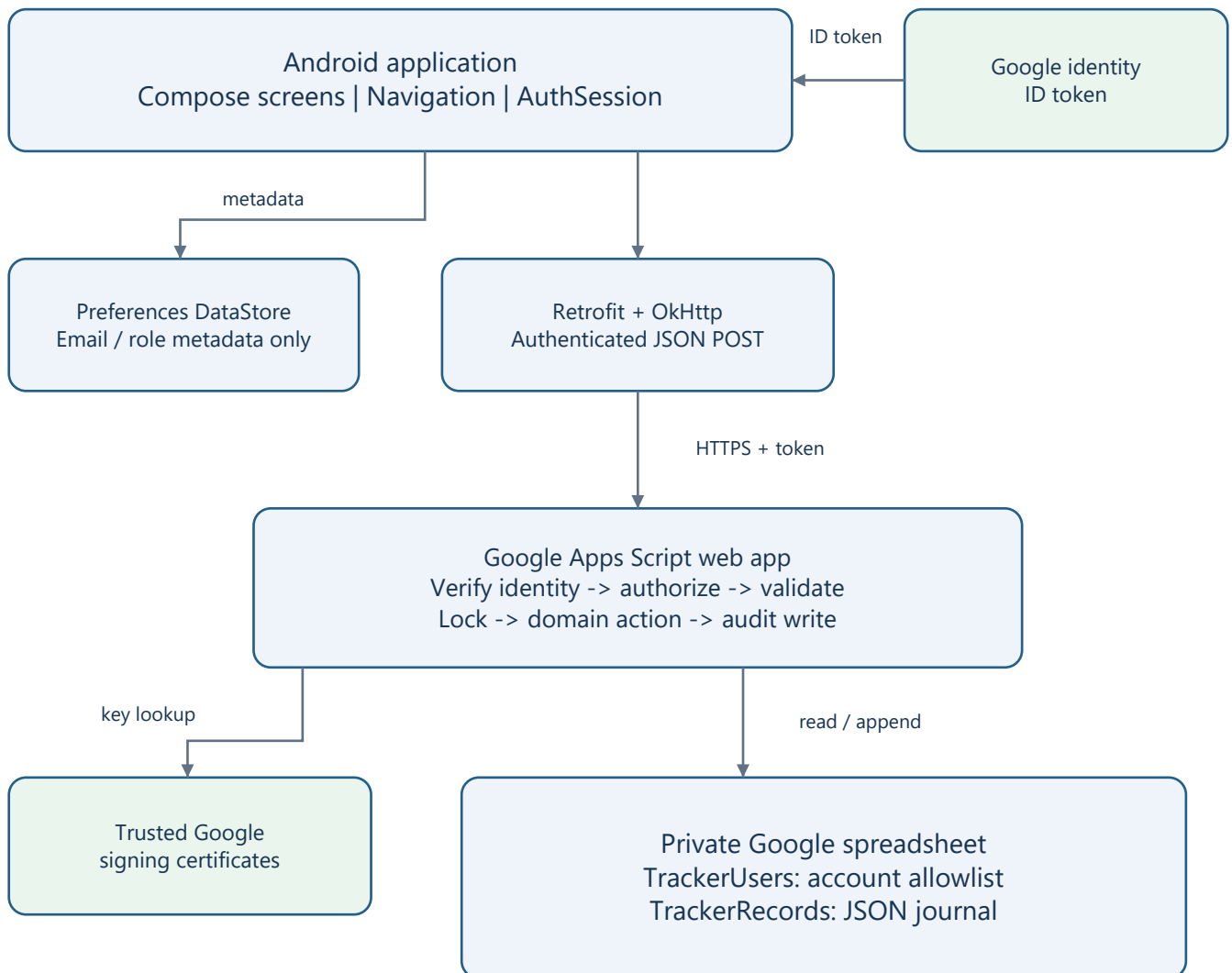
Compose LaunchedEffect and rememberCoroutineScope coordinate loads and user actions.

CancellationException is rethrown. AuthSession exposes a read-only StateFlow. NetworkModule manually constructs one lazy Retrofit service; the baseline uses no dependency-injection framework or background worker.

3.4 Design rationale

Sheets and Apps Script meet the compulsory backend requirement. Central domain authorization prevents UI-only permission enforcement. Journal versions preserve evidence, but reconstructing the entire journal per request creates a scaling limit. A future repository/ViewModel layer and recoverable current-state projection should improve maintainability and capacity without changing the mandated backend.

3.5 System architecture



Trust boundary: device data is untrusted; backend checks every action.

Figure 1. System architecture

The device is outside the backend trust boundary. Google verifies identity issuance; Apps Script verifies the token and applies the account/subject scope before storage access. Device preferences are metadata only.

3.6 Component dependency view

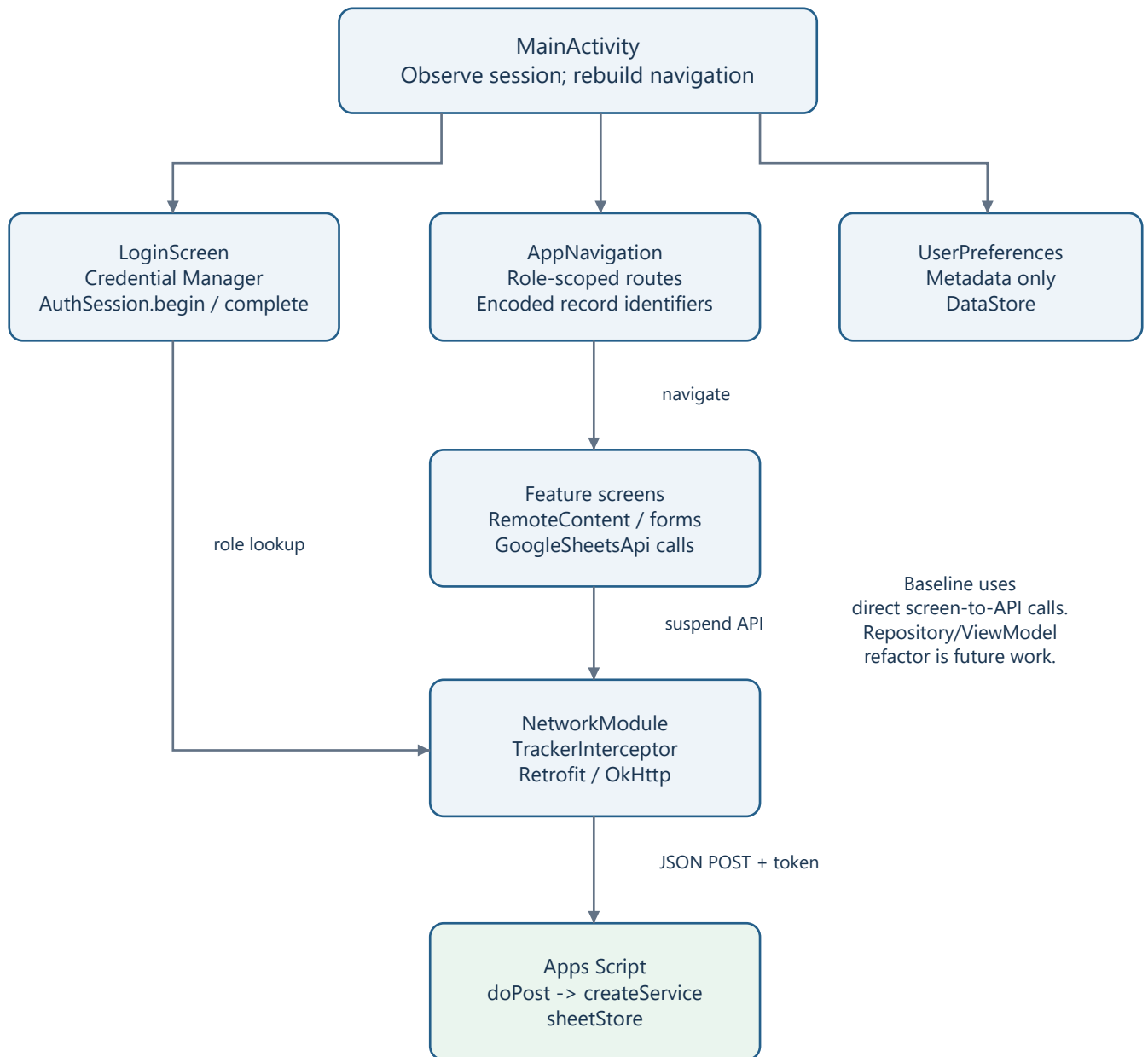


Figure 2. Component dependency view

Solid arrows show the main baseline dependencies. Shared screen helpers wrap direct suspend API calls. A Repository, Room cache and WorkManager queue are proposed future additions rather than existing components.

4. Component design - identity and management

4.1 Entry and session component

MainActivity creates the Compose root, applies the fixed dark theme, observes AuthSession.user and schedules expiry cleanup. key(email, role) rebuilds navigation for an identity change. StudentTrackerApp is a minimal Application extension point, not a dependency container.

4.2 Authentication component

LoginScreen checks configuration, asks Credential Manager for a Google ID token and calls AuthSession.begin. getUserRole verifies identity on the server; the client checks success, supported role and future expiry, stores email/role metadata, then calls AuthSession.complete. Cancellation/failure clears the temporary token and provides user feedback.

4.3 Session storage component

SignedInUser contains email, name, role and expiresAt. AuthSession holds the volatile token and MutableStateFlow privately. UserPreferences stores only email/role in Preferences DataStore; clearSession removes metadata and clears AuthSession. There is no stored refresh token or automatic authenticated restore.

4.4 Administrator and directory components

AdminDashboardScreen loads real counts. DirectoryScreen adapts profile records into stable IDs, titles and subtitles, with case-insensitive search and keyed list rows. Student/Faculty/Subject list screens provide entity-specific loaders and navigation callbacks.

4.5 Profile and subject editors

Add/Edit screens collect required and optional fields, perform frontend checks and submit typed requests. Record detail/edit destinations fetch the current record by encoded ID. Revisions protect updates and deactivation. Server checks remain authoritative for duplicates, active references, capacity and permissions.

4.6 Shared interaction components

WorkspacePage supplies the Scaffold/top bar; RemoteContent models loading, failure, retry and successful content. guardedBack blocks leaving during supported writes and confirms discarding dirty forms. rememberSaveable provides supported configuration-change retention; it is not a durable offline database.

4. Component design - academic and backend

4.7 Academic hub and enrollment

AcademicHubScreen lists active authorized subjects; FacultyDashboardScreen delegates to it. EnrollmentScreen loads students and the roster revision, edits a saveable selected-ID set and submits EnrollmentRequest. The server resolves active students and enforces a distinct roster of at most 250 IDs.

4.8 Attendance component

AttendanceScreen loads minimal roster entries, maintains date/slot/status JSON, supports mark-all-present and confirms a complete roster. It derives a UUID from a payload fingerprint for matching retries. createService validates the roster/date/statuses and appends an immutable session snapshot keyed by subject/date/lowercase-slot.

4.9 Assessment component

AssessmentsScreen lists assessments and opens an editor. New assessments use stable UUID identifiers; mark entries distinguish pending, absent and scored values. Draft saves include expected revision and request ID. Publication requires no pending marks and locks the stored published record.

4.10 Progress and reports

StudentDashboardScreen calls getMyProgress without accepting an arbitrary student identity. AttendanceReportScreen shows a scoped subject snapshot and current fixed 75% shortage indicator. CsvExporter quotes cells, escapes quotes, preserves Unicode through UTF-8 BOM/CRLF and neutralizes spreadsheet formula prefixes.

4.11 Backend components

Source component	Key design responsibilities
auth.js	JWT parsing, RS256 verification, trusted key constraints and error masking.
core.js	validateIdentity, principalFor and createService; domain authorization, rules, calculations and retry/conflict checks.
apps-script.js	doPost, doGet refusal, certificate caching, Script Properties, lock handling, setupTracker and sheetStore.
scripts/build.mjs	Bundle pinned RSA library + application source and generate manifest/license artifacts.

5. Data design - storage and concurrency

5.1 Storage design

The live cloud data model has two tabs: TrackerUsers and TrackerRecords. Academic entities are serialized JSON snapshots in an append-only journal, not SQL tables or separate entity sheets. No Room database or encrypted local academic database exists in the baseline.

Tab / columns	Design meaning
TrackerUsers	email, role, profileId, name, active, googleSubject. Exactly one active matching account is required. Faculty/student profileId links to an active domain record.
TrackerRecords	kind, id, data, at, actor, action. id and actor are JSON-encoded; data contains the entity snapshot. Cells are written with text format.

5.2 Projection and mutation algorithm

- Read journal rows in order and reconstruct latest values by (kind, decoded id).
- Authenticate before academic reads; resolve the allowlist and role under the lock.
- Check current record revision or attendance uniqueness; validate active references and complete rosters.
- Append the next JSON snapshot with verified actor, action and timestamp; update the request-local Map.
- Flush spreadsheet changes and release the lock in finally. Return a success or typed error envelope.

5.3 Concurrency and idempotency

ScriptLock.tryLock uses a 10-second acquisition window and serializes supported operations, including reads. Revision checks protect profiles, enrollment and assessments. Identical supported retries are recognized; reusing a request ID with different assessment content returns conflict. The lock does not create a rollback-capable multi-operation database transaction.

5.4 Bounds and growth

HTTP bodies are limited to 100,000 characters, tokens to 16,000 characters and saved aggregates to 40,000 characters. Enrollment allows at most 250 students per subject. Actual payload size can impose a lower practical capacity. Full-journal projection scans require a measured production optimization plan.

5. Data design - entity schema and calculations

5.5 Logical entity inventory

Entity / key	Fields stored in JSON snapshot
Student / rollNumber	name, email, phone, department, batch, year, semester, section, parentName, parentPhone, parentEmail, active, revision, updatedAt.
Faculty / employeeId	name, email, phone, department, designation, joiningDate, active, revision, updatedAt.
Subject / subjectCode	subjectName, facultyId, semester, totalClasses, active, revision, updatedAt.
Enrollment / subjectCode	studentIds[], revision, updatedAt.
Attendance / subject-date-slot tuple	subjectCode, date, slot, requestId, records[] {studentId, name, status}, updatedAt.
Assessment / assessmentId	subjectCode, title, maxMarks, marks[] {studentId, name, status, score}, published, revision, requestId, updatedAt.

5.6 Calculation design

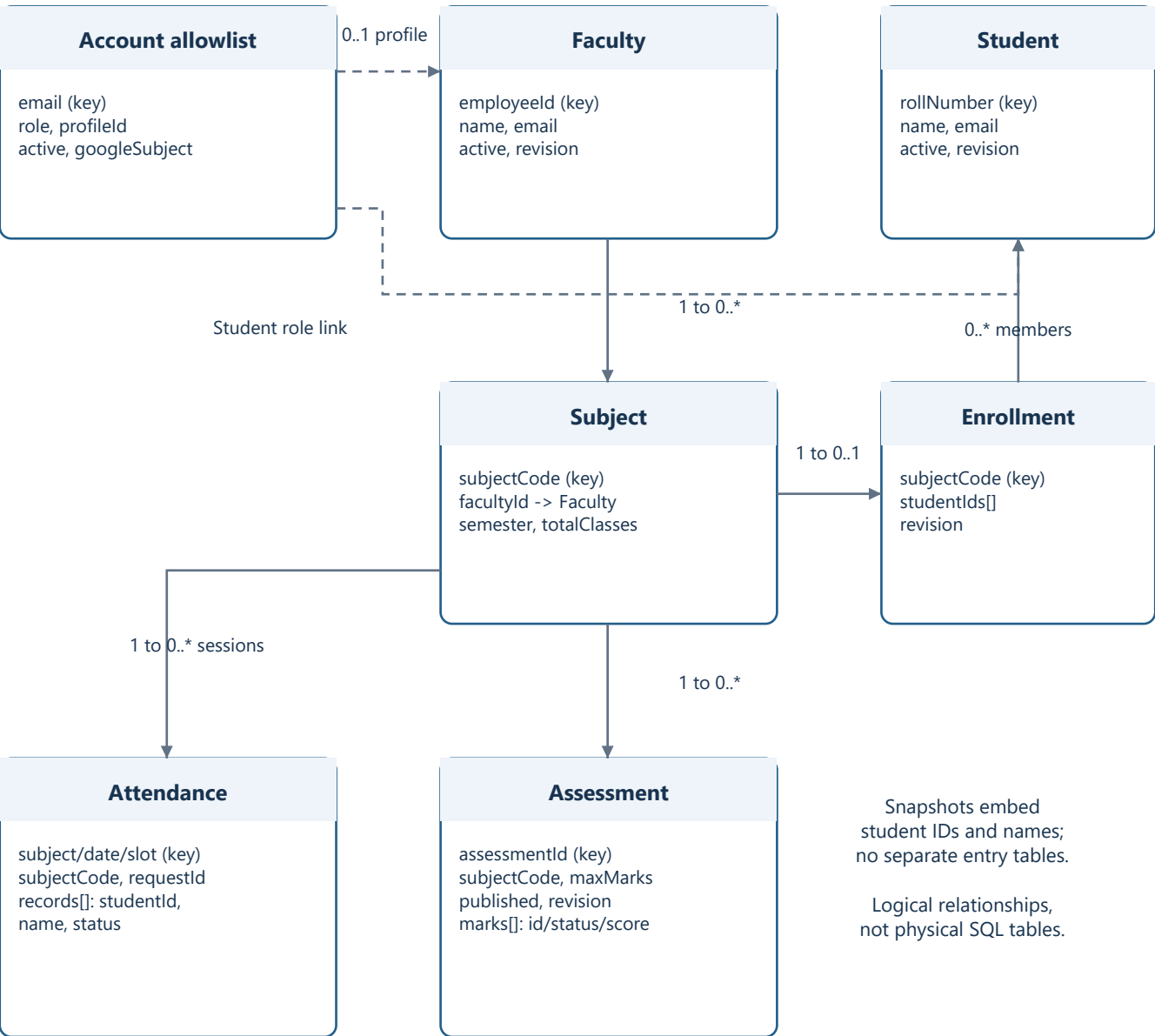
Attendance denominator includes present, late and absent entries from snapshots containing the student. Numerator includes present and late; excused is excluded. $\text{percentage} = 100 \times \text{numerator} / \text{denominator}$, displayed to one decimal. Denominator zero returns an empty percentage interpreted as N/A. Planned totalClasses is not substituted as the denominator.

Maximum marks is 1..10000; scored marks are 0..maximum. Zero is a real score. Absent/pending have null score. Publication excludes pending entries. Final weighted grades, exemptions and retest policies require additional design and institution decisions.

5.7 Archive and schema evolution

Supported deactivation sets active=false and preserves history. Archived IDs cannot be reused. A production migration mechanism is not yet implemented; proposed terms, corrections and indexed projections must have schema versions, tested migrations and rollback compatibility.

5.8 Logical entity relationship diagram



All academic entity versions live in TrackerRecords JSON journal rows.

Figure 3. Logical entity relationship diagram

The figure describes logical relationships only. Embedded roster and marks arrays preserve historical student IDs/names; direct spreadsheet operators still need restricted access. Physical persistence remains the two-tab journal design.

6. Interface design - transport and reads

6.1 Authentication and network contract

Credential Manager obtains a Google ID token for GOOGLE_WEB_CLIENT_ID. All actual academic requests are HTTPS JSON POST to the deployment /exec path. Some Retrofit read methods are declared @GET with @Query, but TrackerInterceptor copies query fields into the JSON body, removes the query string and adds idToken before transmission.

The Web client ID is an audience identifier, not a service-account secret. The app does not call the Google Sheets REST API directly and does not use a Cloudflare role service. Apps Script uses its configured owner permissions to access the spreadsheet.

6.2 Read action inventory

Action	Caller scope / response fields
getUserRole	Approved account -> email, name, role, expiresAt.
getDashboard	Admin -> dashboard {students, faculty, subjects, departments}.
getStudents / getStudent	Admin -> students[] / student; detail key rollNumber.
getFaculty / getFacultyMember	Admin -> faculty[] / facultyMember; detail key employeeId.
getSubject	Admin -> subject; key subjectCode.
getSubjects	Admin: all active; faculty: assigned; student: enrolled. Response subjects[].
getRoster	Admin or assigned faculty -> students[] {studentId,name}, revision.
getAttendanceReport	Admin or assigned faculty -> report[] with present/count/percentage.
getAssessments	Admin or assigned faculty -> assessments[] for subject.
getMyProgress	Student only -> own attendance[] and results[].

6.3 Envelope rules

Success responses contain status="success", apiVersion=2 and action-specific fields. Failures contain status="error", apiVersion=2, code and message. Apps Script domain errors are expressed in JSON; the client must inspect the envelope even when HTTP transport succeeds. doGet rejects data requests.

6. Interface design - writes and examples

6.4 Mutation inventory

Action	Required contract / scope
add/update/delete Student, Faculty, Subject	Administrator. Typed profile fields; update/delete expected revision. Delete deactivates rather than removes.
saveEnrollment	Administrator. subjectCode, studentIds[], revision.
saveAttendance	Admin or assigned faculty. subjectCode, date, slot, requestId, records[] {studentId,status}.
saveAssessment	Admin or assigned faculty. assessmentId, subjectCode, title, maxMarks, marks[], published, revision, requestId.

6.5 Example attendance request - illustrative data

```
{
  "action": "saveAttendance",
  "idToken": "<runtime Google ID token>",
  "subjectCode": "DEM0101",
  "date": "2026-10-01",
  "slot": "Period 1",
  "requestId": "<stable request UUID>",
  "records": [
    {
      "studentId": "DEMO-S001",
      "status": "present"
    }
  ]
}
```

The example is valid only if the authorized subject roster contains exactly that student. It is not a live credential or claim that this demo session has been submitted.

6.6 Example conflict response

```
{
  "status": "error",
  "apiVersion": 2,
  "code": "CONFLICT",
  "message": "The class roster changed. Reload it before submitting."
}
```

Internal interface: suspend Retrofit methods deserialize typed data classes; TrackerApiException carries a code and readable message. The backend store contract exposes get(kind,id), list(kind) and put(kind,id,data,audit), allowing domain tests to replace Sheets with an in-memory store.

6. Interface design - screens and exports

6.7 Screen responsibilities

Screen group	Interaction design
Login	Google sign-in button, configuration failure and cancellation feedback.
Administrator dashboard	Active counts and management/academic/report navigation.
Directories / profile forms	Searchable stable-ID records; required fields; read-current edit; revision-aware save/deactivation.
Academic hub / enrollment	Authorized subject cards; administrator selection of enrolled students.
Attendance	Date/session fields, status chips, completion count, mark-all-present, confirmation and submitted state.
Assessment editor	Title/maximum, pending/absent/scored entries, draft save and explicit publish.
Student dashboard	Personal attendance indicators and published result cards; refresh/logout.
Attendance report	Subject snapshot, fixed threshold indication and CSV share action.

6.8 Navigation and form state

AppNavigation constructs only the current role graph. Routes encode immutable record identifiers with Uri.encode and reload details rather than using a global selected-object cache. MainActivity keys the graph by email/role. Saveable form values support configuration changes; busy writes and dirty forms use guardedBack.

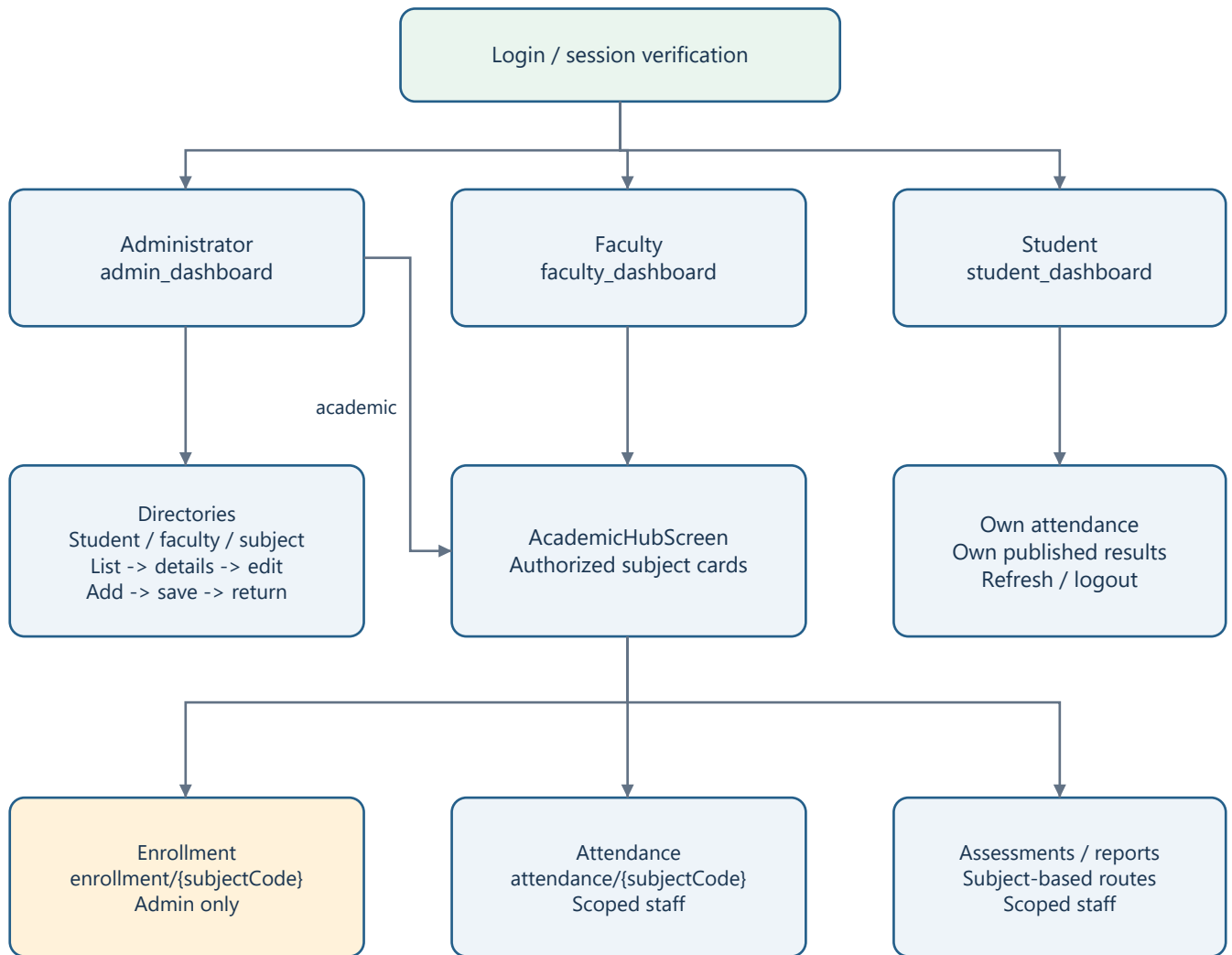
6.9 Export interface

CSV output uses UTF-8 BOM, CRLF, quoted cells and escaped embedded quotes. Formula-like prefixes are neutralized. AttendanceReportScreen writes sanitized filenames under the cache export directory and shares through FileProvider with a temporary read grant. Cleanup removes export files older than 24 hours when cleanup runs.

6.10 Accessibility and UI limits

The current theme is fixed dark and many labels are hard-coded. Large fonts, TalkBack, smaller screens, landscape and keyboard/system-bar interactions still require acceptance testing. String resources, consistent field-level feedback, filter/scroll restoration and optional theme selection are future improvements.

6.11 Role-based navigation diagram



Logout or expiry clears the session and rebuilds the graph at login.

Figure 4. Role-based navigation diagram

Shared staff routes are shown under the academic hub. Enrollment is administrator-only. The faculty hub receives backend-filtered subjects; student navigation exposes only the personal dashboard. The server independently checks every action.

7.1 Sign-in and role-resolution sequence

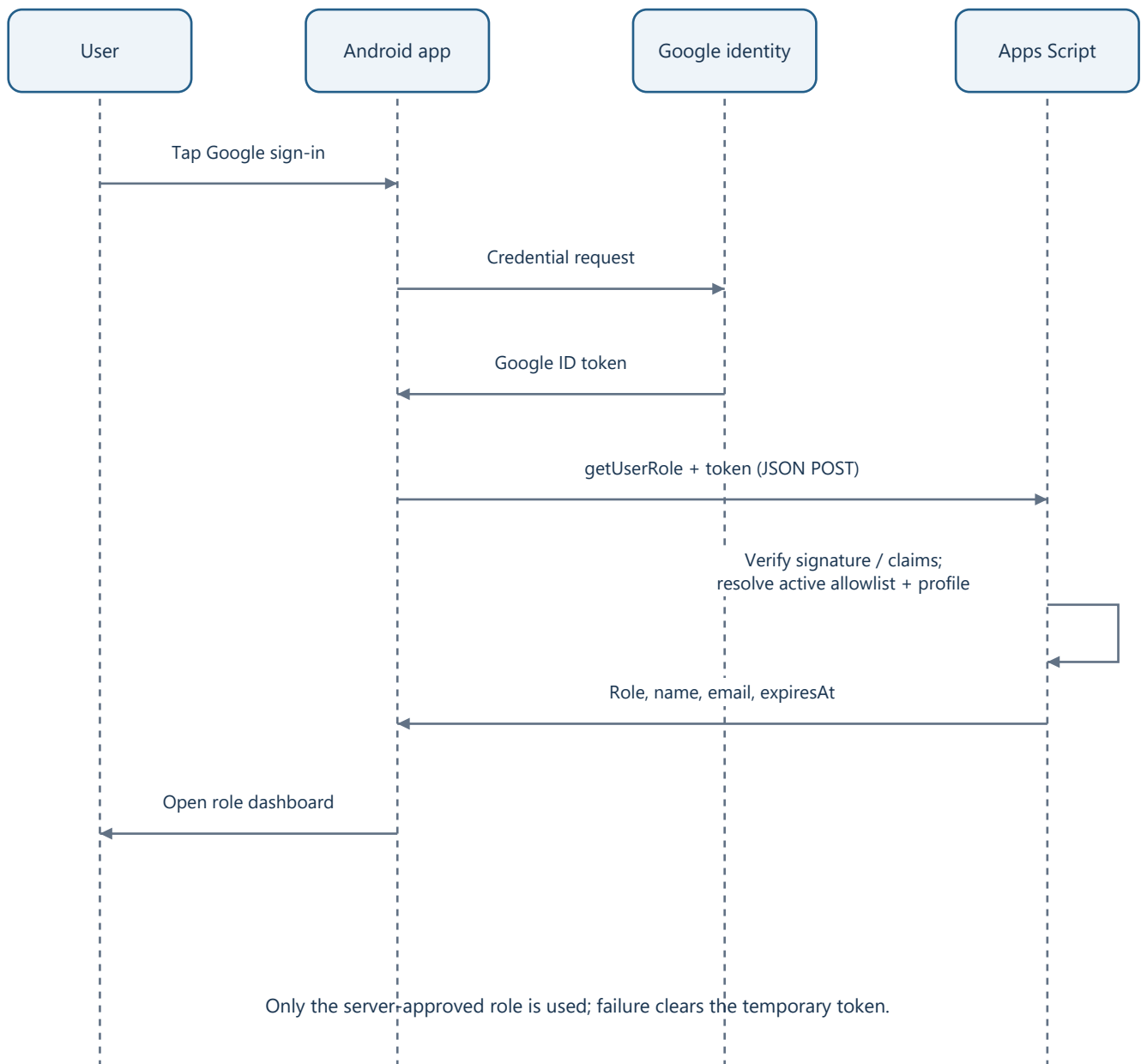


Figure 5. Sign-in and role-resolution sequence

Credential issuance and application authorization are separate. The app begins a temporary token session, calls `getUserRole` and completes `AuthSession` only after a supported, unexpired server response. A fresh process repeats sign-in.

7.2 Attendance runtime flowchart

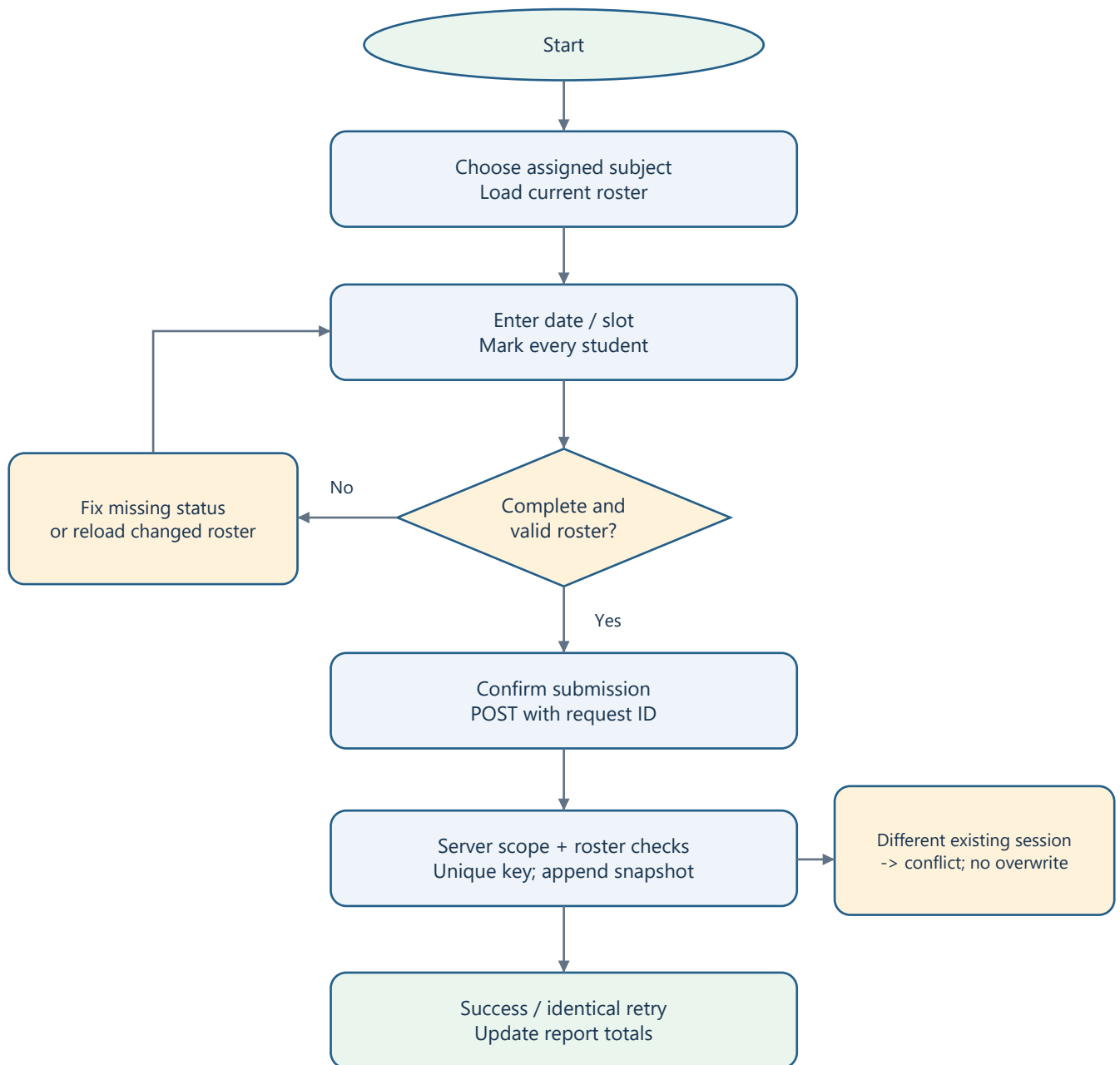


Figure 6. Attendance runtime flowchart

Attendance uses the current minimal roster. The payload fingerprint generates a stable retry identity; the server snapshots names/statuses and applies the unique session key. Roster changes and competing submissions return conflicts.

7.3 Assessment runtime flowchart

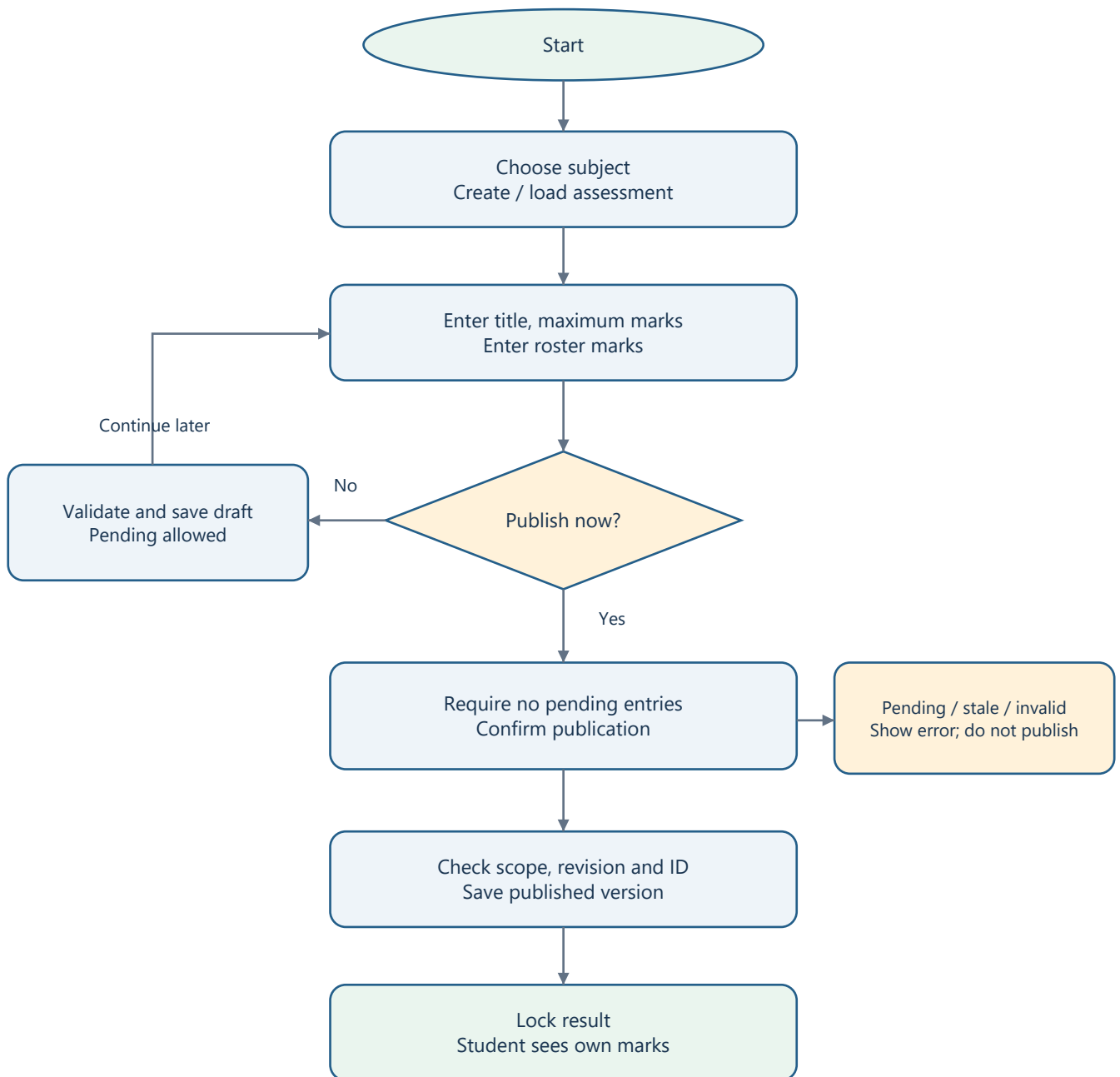
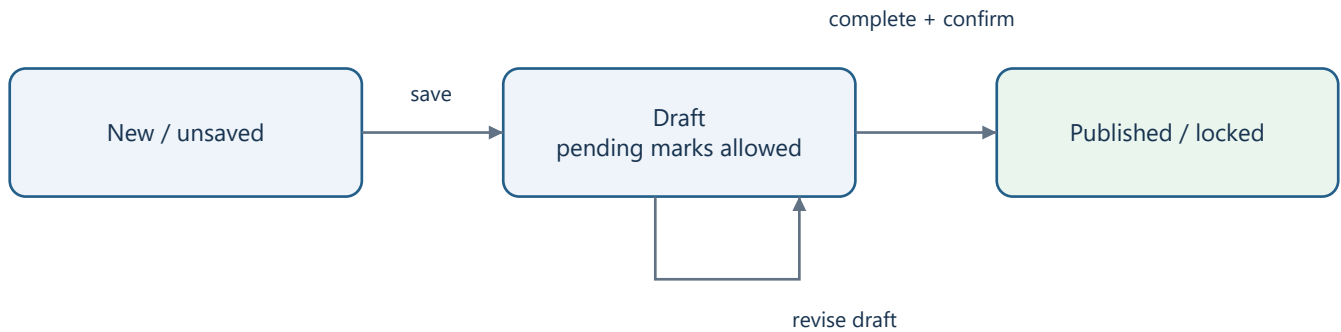


Figure 7. Assessment runtime flowchart

Drafts may contain pending marks. A publish request is validated against the complete current roster and expected revision. Published marks are read-only in the baseline and visible to the relevant student.

7.4 Attendance and assessment state transitions

Assessment lifecycle - implemented baseline



No direct Published -> Draft transition exists in the baseline.

Attendance session lifecycle - implemented baseline



Different content for an existing subject/date/slot is a conflict.

Proposed extension: correction request -> approval -> new audited version

Figure 8. Attendance and assessment state transitions

States are conceptual descriptions of implemented fields/UI behavior. The baseline has no direct correction or unpublish path. The proposed correction extension must retain original and approved replacement evidence.

8. Security design

8.1 Authentication trust boundary

verifyGoogleToken accepts RS256 only, selects trusted Google signing material by kid and requires an RSA key of at least 2048 bits with exponent 65537. It verifies the signature and then issuer, audience, expiry, issued-at, optional not-before, subject and verified email. Cached Google certificates respect a bounded cache lifetime; an unknown key triggers one refresh.

8.2 Authorization design

principalFor requires exactly one approved active TrackerUsers record and a supported role; an optional googleSubject pins the Google identity. An optional ALLOWED_GOOGLE_DOMAIN compares the signed hosted-domain claim. Non-admin actions require an active profile; subjectFor limits faculty to their assigned subjects; getMyProgress derives student scope from the verified principal.

8.3 Device and transport protection

Tokens exist only in memory. DataStore metadata cannot establish authorization. Manifest cleartext traffic is disabled and backup is disabled with storage exclusions. NetworkModule adds tokens only to JSON request bodies; exports contain report data rather than credentials. These controls are not a claim of certificate pinning or encrypted local academic storage.

8.4 Backend and spreadsheet protection

Validate body/token/field lengths before domain processing. Sheet journal cells use text format and JSON encoding to reduce unintended spreadsheet interpretation. Internal exception details are replaced by a generic client message. The public web-app endpoint still requires token verification and allowlist approval for all academic data.

8.5 Production security work

Restrict operator spreadsheet sharing; establish cloud ownership and administrator recovery; add audited account/role management, redacted diagnostics, abuse controls and institution privacy/retention rules. Audit journal behavior is application-enforced and does not make the spreadsheet immutable to its owner.

9. Error handling and reliability

9.1 Failure categories and user response

Failure code / condition	Handling design
UNAUTHENTICATED	Clear AuthSession; require sign-in; do not trust cached metadata.
FORBIDDEN	Show access denial; do not retry as another role or relax permissions.
VALIDATION / NOT_FOUND	Show actionable input/record message; user corrects or reloads.
CONFLICT	Reload current version/roster and review; never silently overwrite.
BUSY / UNAVAILABLE	Show temporary failure and allow controlled retry.
CAPACITY	Explain supported size limit; do not truncate academic records.
CONFIGURATION	Require matching deployment/audience/schema setup; refuse legacy API.
HTTP / INVALID_RESPONSE	Show service/deployment failure; close failed response.
INTERNAL	Expose a generic failure message; no internal stack trace in API response.
Coroutine cancellation	Rethrow cancellation rather than turning it into successful content.

9.2 Timeouts and retries

OkHttp uses a 20-second connect timeout, 60-second read timeout and 75-second call timeout. Automatic connection retry is disabled. Server idempotency protects matching supported resubmissions, but there is no persistent client submission queue or background retry worker.

9.3 Data consistency and uncertainty

An HTTP timeout can occur after a server write. Current workflows may allow an identical supported retry; durable submission status lookup/recovery remains planned. Revision checks and locking avoid silent competing updates, but multi-record operations need a checkpoint/partial-failure design before they are added.

9.4 Recovery limitations

Forms may survive configuration changes; persistent offline draft recovery and term-level correction workflows are not implemented. Backup/restore, quota-aware operation, monitoring and release rollback must be established before institution-wide reliance.

10. Deployment and configuration

10.1 Build environment

Use Android Studio or the Gradle wrapper with JDK 21 and required Android SDK packages. app/build.gradle.kts configures minSdk 24, target/compile 37, Java compatibility 11 and app version 1.0. Node.js 22+ builds/tests the Apps Script bundle with the pinned node-forge dependency.

10.2 External configuration

Configuration	Location / purpose
TRACKER_BASE_URL	External Gradle property/environment; deployment URL without exec, ending in /.
GOOGLE_WEB_CLIENT_ID	Android build value and matching Script Property; token audience.
SPREADSHEET_ID	Script Property selecting the private institution spreadsheet.
ALLOWED_GOOGLE_DOMAIN	Optional Script Property; validate signed Google hosted-domain claim.
Script timezone / runtime	Generated appsscript.json: Asia/Kolkata; V8.
Script OAuth scopes	Spreadsheet access and external requests for certificate retrieval.
Android OAuth client	Register application package and actual debug/release signing SHA-1.

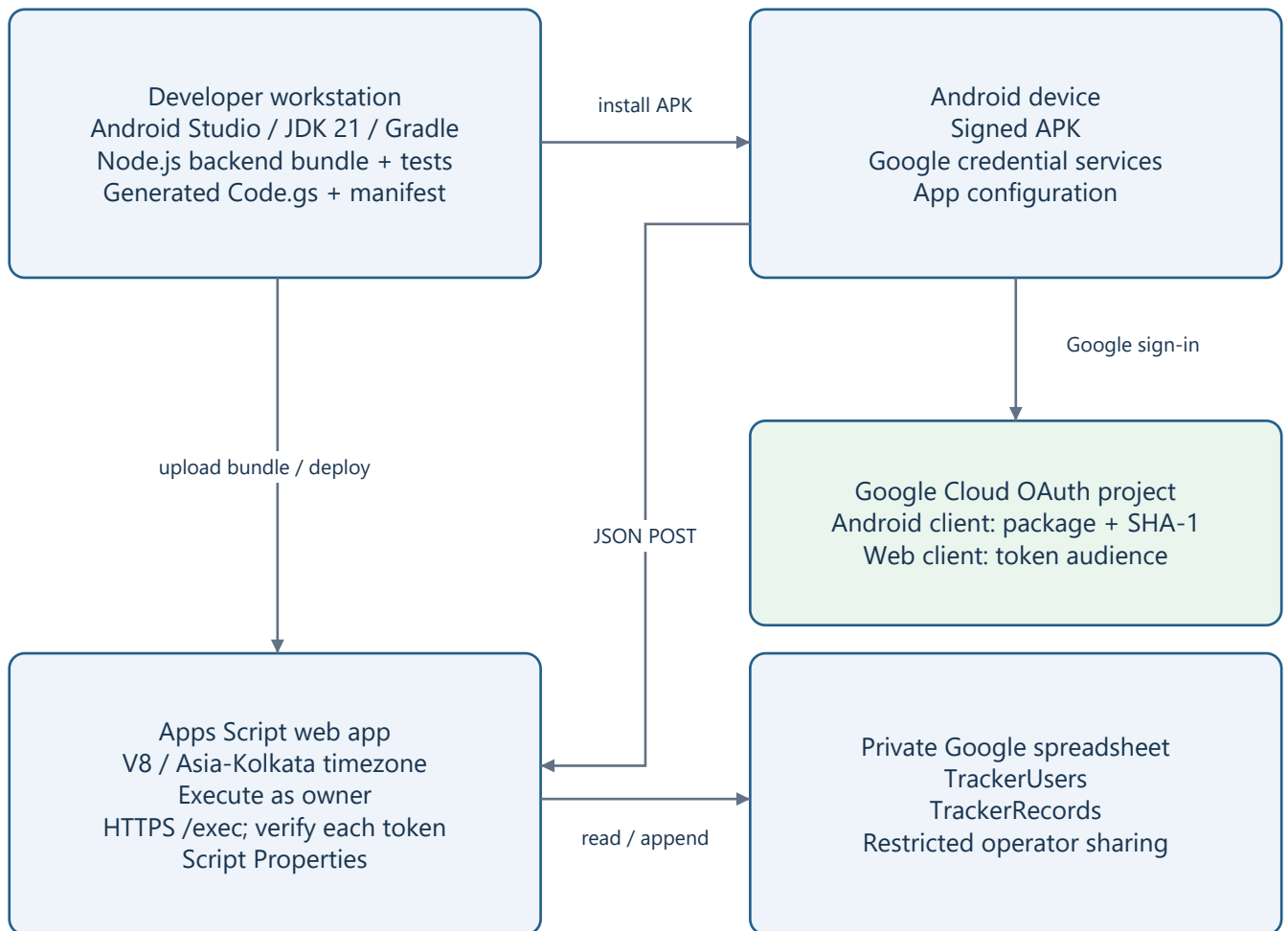
10.3 Deployment sequence

Build the backend; upload dist/Code.gs and appsscript.json; set properties; run setupTracker and approve required scopes; provision approved account mappings; deploy the web app executing as owner. Where institution policy permits Anyone access, backend authentication remains mandatory. Configure and rebuild the Android client, then run all-role acceptance on a real device.

10.4 Current setup and release boundary

Demo OAuth values, deployment URL and spreadsheet are configured, initialization succeeded and administrator/faculty/student mappings exist. The debug APK was rebuilt with real configuration. Live sign-in/full workflow and production signing are not yet verified. Production needs distinct environments, institutional ownership, secure release key handling and rollback evidence.

10.5 Deployment topology



Proposed production topology: separate dev, staging and production copies.

Public web-app reachability does not grant permission to academic data.

Figure 9. Deployment topology

Developer artifacts are deployed to a Google Apps Script web app and installed as an Android APK. OAuth registration links the package/signing certificate to identity issuance; Apps Script owns authorized access to the private spreadsheet. No production credentials are printed here.

11. Testing and verification

11.1 Recorded local verification

Previous verification records show 27 backend tests passed and 9 Android JVM tests passed, plus successful debug assembly and lint with zero errors and 33 warnings. The count includes existing template tests. This SDD creation does not constitute a new app test run.

Backend tests exercise independently signed RSA tokens, claims, allowlists, domain permissions, calculations, revisions, retry handling and mocked Apps Script/Sheets operations. Android tests cover CSV handling and authenticated network transport/failures. Mock tests do not prove live Google identity or quota behavior.

11.2 Acceptance test plan - outstanding live evidence

Test ID	Scenario / expected result
SDD-T01	Google approved sign-in and cancellation; invalid/inactive accounts denied; correct role routing.
SDD-T02	Admin creates/edits profiles/subject and enrollment; duplicates and stale revisions rejected.
SDD-T03	Assigned faculty submits full roster; identical retry gives one session; unassigned request denied.
SDD-T04	Calculate present/late/absent/excused and N/A correctly; preserve historical roster eligibility.
SDD-T05	Draft pending allowed; publish pending denied; zero/absent correct; published edit denied.
SDD-T06	Student sees own attendance/marks and no other student/draft data.
SDD-T07	CSV opens on phone with Unicode, quotes and safe formula handling.
SDD-T08	Rotation, process restart, logout/account change and network loss produce accurate state.
SDD-T09	Concurrent edits, lock contention, lost response and realistic journal growth tested.
SDD-T10	Release signing, accessibility, backups/restore, migrations and rollback verified before production.

Record actual device/environment, date, expected/actual result and logs/screenshots when executing this plan. The template instrumented test is not an end-to-end acceptance suite.

12. Requirements traceability - baseline

Each suffix below expands to CUTM-SRS-SAPT-XX in the project SRS. Built means implementation evidence exists; it does not mean every listed action passed live device acceptance.

SRS ID	Requirement	Design / source evidence
01	Launch	MainActivity / AppNavigation
02	Google sign-in	LoginScreen / auth.js
03	Role and profile authorization	principalFor / subjectFor / getMyProgress
04	Expiry and logout	AuthSession / UserPreferences / MainActivity
05	Admin dashboard	AdminDashboardScreen / getDashboard
06	Student management	Student screens / core.js CRUD
07	Faculty management	Faculty screens / core.js CRUD
08	Subject management	Subject screens / active faculty validation
09	Safe deactivation	CRUD delete branch / active=false
10	Enrollment	EnrollmentScreen / saveEnrollment
11	Faculty workspace	AcademicHubScreen / getSubjects
12	Attendance capture	AttendanceScreen / validateRoster
13	Attendance persistence	saveAttendance / unique key / retry check
14	Attendance calculation	attendanceReport / snapshot aggregation
15	Assessment drafts	AssessmentEditor / saveAssessment
16	Assessment publication	published guard / no-pending validation
17	Student progress	StudentDashboardScreen / getMyProgress
18	Reports / CSV	AttendanceReportScreen / CsvExporter
19	Loading / retry / form protection	RemoteContent / guardedBack / saveable state
20	Audit / conflicts	sheetStore / ScriptLock / revision checks

12.1 Consistency with the SRS

Both documents describe Compose, minSdk 24, direct screen-to-API calls, Google token verification, an online baseline and two Sheets tabs. Neither claims Room/Hilt/WorkManager, configurable grading or approved published-record corrections as implemented.

12. Requirements traceability - planned scope

SRS ID	Status	Required design extension
21	Planned	Account-management screens; provisioning/suspension, audited role changes and last-admin recovery safeguards.
22	Planned	Terms/programs/sections/subject offerings; dated enrollment; configurable effective-dated attendance and grade policies.
23	Planned	Correction request lifecycle, approval permissions and before/after audited versions for attendance/marks.
24	Planned	Persistent account-isolated drafts/queue, stable operation IDs, permission revalidation and conflict recovery.
25	Planned	Import templates/preview/errors; term/section/date report filters; printable progress reports.
26	Planned	Separate environments, backups/restore, capacity/monitoring, privacy ownership and signed release acceptance.
27	Optional	Separately scoped QR/location attendance, communication, timetable, assignments, guardian access or analytics.

12.2 Non-functional design mapping

Quality	Mechanism / remaining evidence
Security and privacy	Signed identity, server scoping, token memory, narrow exports; live denial/retention/abuse tests still required.
Correctness and integrity	Roster validation, status semantics, revisions, audit journal and matching retries; device/concurrency acceptance pending.
Performance and capacity	Request bounds and serialized access exist; p95/load targets need measurement and projection/pagination work.
Recovery and availability	Backup restore, monitoring, operator response and durable draft recovery are production work.
Accessibility and maintainability	Shared UI and isolated domain store help; device accessibility and CI/refactor work remain.

13. Limitations and future enhancements

13.1 Known baseline limitations

- Live Google sign-in, deployed academic operations and real-device acceptance are not yet established.
- No durable offline academic database, background sync, refresh-token persistence or automatic authenticated process restart.
- No first-class academic term/section/subject offering model or dated enrollment lifecycle.
- Attendance shortage is currently fixed at 75%; weighted grades, retests and institution policies are not implemented.
- Submitted attendance and published marks are locked without an approved correction workflow.
- Allowlist account administration remains an operator task rather than an in-app workflow.
- Full-journal scans, global locking and bounded aggregates limit capacity; no backend pagination/compaction is implemented.
- Production signing, automated backup restore, monitoring, migration/rollback and complete accessibility evidence remain outstanding.

13.2 Proposed improvements

Prioritize live demo acceptance, account/academic lifecycle and corrections, then durable drafts, bulk import, filtered reports and capacity improvements. Refactor presentation/data boundaries where needed for testable state and synchronization. All release gates must pass before production sign-off.

13.3 Optional scope

Consider notifications, timetable, QR/location assistance, assignments/Drive attachments, guardian access, delegated roles, analytics and theme/language choices only after explicit institution scope decisions. Each extension needs its own authorization, privacy, dependency and acceptance design.

14. Conclusion and references

14.1 Design conclusion

Student Tracker provides a connected Android design for administrator preparation, scoped staff attendance/assessment entry and personal student progress. Google Apps Script authenticates and authorizes requests before applying domain rules and appending JSON snapshots to Google Sheets. Revision and retry semantics protect supported writes from silent duplication or stale replacement.

The baseline is suitable for further demonstration and acceptance work, with recorded local verification and configured demo resources. Production readiness remains dependent on live all-role testing, academic lifecycle/policy/correction features, recovery and operational controls. This SDD records those limits so the design, SRS and implementation remain aligned.

14.2 Referenced material

- TeamPulse_SDD 1.pdf, 19 pages: supplied structure/layout reference; sample names, guide and technology choices are not inherited.
- Student_Tracker_SRS.pdf / docs/Student_Tracker_SRS.md: CUTM-SRS-SAPT-VER-0.1 requirements baseline.
- docs/FEATURE_CHECKLIST.md and PRODUCTION_PLAN.md: implementation and production backlog.
- docs/DEMO_SETUP_STATUS.md and docs/STAGING_CHECKLIST.md: deployment evidence and remaining live acceptance.
- Android source under app/src/main; app/build.gradle.kts; gradle/libs.versions.toml; backend/src and backend/test.
- backend/README.md: deployment procedure. Its initial undeployed description is superseded by DEMO_SETUP_STATUS.md.

14.3 Document control and review

Version / date	Change / review state
0.1 / 1 October 2026	Initial project SDD; seven team members included; architecture, runtime, state and deployment diagrams; SRS traceability.
Review status	Team/institution review draft. Guide identity, member task allocation and production sign-off are not asserted.

Appendix A. Source component index

Paths below are relative to the repository root. Android package root is `app/src/main/java/com/example/studenttracker/`. This index supports code explanation and links design responsibilities to implementation.

Location	Responsibility
MainActivity.kt / StudentTrackerApp.kt	Activity/session-driven composition and application entry point.
presentation/navigation/AppNavigation.kt	Role graphs; URI-encoded ID routes; remote detail loads.
presentation/auth/LoginScreen.kt	Google credential acquisition and role handshake.
data/network/AuthSession.kt / NetworkModule.kt	Memory session, token POST interceptor, typed failures, lazy Retrofit.
data/network/GoogleSheetsApi.kt / AcademicModels.kt	Profile API contracts and academic request/response models.
data/local/UserPreferences.kt	Email/role Preferences DataStore metadata and logout cleanup.
presentation/admin / faculty / student / subject	Dashboards, directories, profile/subject forms and details.
presentation/academic	Academic hub, enrollment, attendance and assessments.
presentation/components	Shared remote loading/layout and unsaved-change guards.
domain/model/CsvExporter.kt; res/xml/file_paths.xml	CSV safety and narrow export file sharing.
backend/src/auth.js / core.js / apps-script.js	Token verifier, business rules, HTTP/Sheets adapter.
backend/test; app/src/test	Backend domain/auth/adapters and Android transport/CSV tests.
backend/scripts/build.mjs; backend/dist	Bundle generation; generated Apps Script code/manifest/license.

No production credentials, keystore passwords or private signing material are included in this document. The supplied team list records authorship, not unverified module ownership.